PROJECT REPORT

Development of an Intelligent Gaming Application Using Convolutional Neural Networks (CNN)

CNN-Based Hand-Gesture-Controlled Pac-Man-Style Maze Game

Course: Deep Learning & Reinforcement Learning (BAI 701)

Submitted by

Student Name: Tanmay Singh

USN: [Enter USN]

Semester & Section: [Enter Semester & Section]

Team Number: [Enter Team Number]

Submitted to

Faculty Name: [Enter Faculty Name]

Department of Artificial Intelligence and Data Science

[Enter College Name]

Academic Year: 2026-2027

2. ABSTRACT

Most games are still played with a keyboard or a controller. In this project, a Pac-Man-style maze game is controlled instead by hand gestures shown to an ordinary webcam. A convolutional neural network (CNN) looks at the player's hand and recognises one of four gestures: a closed fist, an open palm, a thumbs-up or a thumbs-down. These steer Pac-Man left, right, up and down.

The network was trained on 2,000 hand images, 500 for each gesture, cropped from the public HaGRID dataset using its own bounding-box annotations and split into 1,600 training, 200 validation and 200 test images. Rather than training a network from scratch, we started from MobileNetV2 pretrained on ImageNet, replaced its 1,000-class output layer with a four-class layer and trained it in two stages: first the new layer on its own, then the last few feature blocks as well. The final model reached 99.5% accuracy on the validation set and 99.0% (198 of 200 images) on the test set, with a macro F1-score of 0.990. On a further 2,000 images of people who are not in the dataset at all, it scored 99.6%.

In the game, recognition runs on its own thread at about 30 frames per second, so the game itself keeps running at 60. A prediction only becomes a command if the model is at least 90% confident and the same gesture wins three of the last five frames; anything less simply produces no command. In live trials, all 80 held gestures were recognised correctly and none of the 40 trials with no hand or a relaxed hand triggered a move. The main weakness is that the model cannot answer "none of these", so an unsupported gesture may still be read as one of the four.

Keywords: convolutional neural network, MobileNetV2, transfer learning, hand gesture recognition, real-time inference, Pygame, human-computer interaction.

TABLE OF CONTENTS

2. ABSTRACT i

3. INTRODUCTION 1

3.1 Background 1

3.2 Problem Statement 1

3.3 Objectives 2

5. SYSTEM REQUIREMENTS 3

5.1 Hardware Requirements 3

5.2 Software Requirements 3

6. PROPOSED SYSTEM 5

6.1 Game Description 5

6.2 Role of CNN 5

6.3 System Architecture 6

7. DATASET DESCRIPTION 7

8. DATA PREPROCESSING 10

9. CNN MODEL DESIGN 12

9.1 CNN Architecture 12

9.2 Explanation of Layers 13

9.3 CNN Architecture Diagram 15

10. MODEL TRAINING 15

11. MODEL PERFORMANCE EVALUATION 18

11.1 Training Accuracy 18

11.2 Validation Accuracy 18

11.3 Test Accuracy 19

11.4 Training and Validation Accuracy Graph 20

11.5 Training and Validation Loss Graph 20

11.6 Confusion Matrix 21

11.7 Sample Predictions....22
12. GAME DEVELOPMENT....23
12.1 Game Interface....23
12.2 Game Logic....24
12.3 CNN-Game Integration....26
13. IMPLEMENTATION....28
14. RESULTS AND DISCUSSION....29
16. CHALLENGES FACED....33
17. LIMITATIONS....34
18. FUTURE ENHANCEMENTS....34
19. CONCLUSION....35
20. REFERENCES....36

3. INTRODUCTION

3.1 Background

Keyboards, mice and gamepads work well, but they are not the only way to play a game. With gesture control the player simply moves a hand, which feels more direct and can help people who find small buttons awkward. The only hardware needed is a webcam, which almost every laptop already has. The hard part is the software: turning a stream of camera images into commands that are right nearly every time.

Convolutional neural networks are a natural fit for this problem. Instead of relying on hand-written rules about finger positions, a CNN learns its own visual features from labelled examples, starting with edges and textures and building up to whole shapes. Training one from nothing usually takes far more images than a student project can collect, but *transfer learning* gets around this. A network that has already learned general features on ImageNet [4], a collection of more than a million labelled photographs, can be adapted to a new task with a few thousand images. We chose MobileNetV2 [3], an architecture designed to run quickly on modest hardware, because the model has to classify webcam frames many times a second while the game is running.

3.2 Problem Statement

The aim of the project is to play a real-time maze game using nothing but hand gestures in front of a normal webcam. That sounds simple, but several requirements pull against each other:

- The four gestures have to be recognised reliably for different people, hands, backgrounds and lighting.
- Recognition has to keep pace with the camera without dragging the game below 60 frames per second.
- When the model is unsure, it should do nothing. A missed command is a small annoyance; a wrong turn into a ghost is not.
- Pac-Man has to turn at the right junction, even though it takes a fraction of a second to be sure what the player's hand is showing.

The work therefore has three parts: preparing a suitable dataset, training a CNN that classifies the gestures accurately, and building it into a playable game in which its output actually decides where Pac-Man goes.

3.3 Objectives

1. Build a balanced dataset of hand images for the four gestures from a documented public source, with a split that anyone can reproduce.
2. Adapt a pretrained MobileNetV2 network to the four gesture classes.
3. Train it with suitable preprocessing and augmentation, using the validation set only to choose the best checkpoint.
4. Measure the final model once on the untouched test set: accuracy, loss, per-class precision, recall and F1-score, and a confusion matrix.
5. Write an original Pac-Man-style maze game in Pygame, with pellets, ghosts, lives, scoring and rounds.
6. Connect the trained model to the game in real time without slowing the game down, and keep the keyboard as a backup.
7. Test the finished application with automated tests and live webcam trials.

5. SYSTEM REQUIREMENTS

Table 1 lists the laptop on which the project was built and tested, next to what the application actually needs. We did not carry out a formal study of minimum hardware, so the right-hand column only contains requirements that follow from the code or from measurements we recorded.

5.1 Hardware Requirements

Table 1: Hardware Used for Development and Hardware Needed to Run the Application

Component	Development and test system (verified)	Needed to run the application
Processor	Intel Core i7-11800H, 2.30 GHz	Any 64-bit processor supported by Python 3.12 and PyTorch
Graphics	NVIDIA GeForce RTX 3050 Ti Laptop GPU, 4 GB VRAM	Optional for playing: without a CUDA GPU the model runs on the CPU. An NVIDIA CUDA GPU is required for training and for the formal evaluation script.
Memory	15.7 GB RAM	Recognition process measured at about 1.0 GB resident memory with CUDA and about 0.67 GB on the CPU; the game alone uses about 48 MB
Camera	Built-in laptop webcam, captured at 640 × 480	Any webcam readable by OpenCV; needed only for gesture control (keyboard-only mode needs none)
Display	Laptop screen (Windows display scaling 125%)	At least 992 × 704 pixels for the game window
Storage	Model file 9.2 MB; dataset images 45 MB	About 2 GB for the CUDA build of PyTorch; the CPU build is smaller

5.2 Software Requirements

The versions in Table 2 were read from the project's virtual environment. They are pinned in *requirements/cuda.txt* and *requirements/cpu.txt*, and the setup script stops if it finds any Python version other than 3.12.

Table 2: Software Environment

Software	Version	Purpose in the project
Operating system	Windows 11 (64-bit)	Development, training and live testing

Software	Version	Purpose in the project
Python	3.12.10, 64-bit	Programming language
PyTorch	2.14.0+cu130	Model definition, training and inference [5]
torchvision	0.29.0+cu130	MobileNetV2 architecture, pretrained weights and image transforms [6]
CUDA runtime / cuDNN	13.0 / 9.24	GPU acceleration
OpenCV (opencv-python)	5.0.0.93	Webcam capture, mirroring, colour conversion and dataset cropping [7]
Pygame	2.6.1 (SDL 2.28.4)	Game window, drawing, keyboard input and sound [8]
NumPy	2.5.3	Array operations on camera frames [11]
Pillow	12.3.0	Image loading for the data pipeline
Matplotlib	3.11.2	Training curves and evaluation figures [12]
scikit-learn	1.9.1	Confusion matrix, precision, recall and F1-score [13]
pytest	(development requirement)	Automated test suite

The setup scripts and the automated tests also run on macOS and Linux in continuous integration. Live play with a webcam, however, was only tested on Windows.

6. PROPOSED SYSTEM

6.1 Game Description

The game is our own version of the arcade classic Pac-Man, written from scratch for this project. The maze layout is defined in the project's code, and everything on screen, from the walls to Pac-Man, the ghosts, the pellets and the bonus fruit, is drawn with Pygame's basic shapes. No sprites or other assets from the original game are used, and the sound effects are generated by the program when it starts.

The rules will be familiar. Pac-Man moves through a maze of 28 columns and 31 rows, eating 330 normal pellets and 4 power pellets while four ghosts hunt him. A power pellet frightens the ghosts for a few seconds, and during that time Pac-Man can eat them for bonus points. The player starts with three lives and loses one each time a dangerous ghost catches Pac-Man. Clearing the board starts a new round with faster ghosts and shorter fright times, and the score and remaining lives carry over. When the last life is gone, the game is over.

Pac-Man can be steered with gestures or with the keyboard (the arrow keys or W, A, S, D), and the player can switch between the two at any moment. The window is 992 × 704 pixels. The maze and its score bar take up the left 560 pixels, and the remaining 432 pixels hold the gesture panel with the live camera view and the recognised command.

6.2 Role of CNN

The CNN is the part of the system that reads the player's hand. For every camera frame it receives the 300 × 300-pixel square in which the player holds their hand, and it outputs a score for each of the four gestures. A score only turns into a direction when the model is confident and the same answer holds over several frames (Section 12.3). The game itself never looks at the camera; it simply receives the directions that come out of this process. Table 3 shows the mapping used everywhere in the project.

Table 3: Gesture-to-Command Mapping

Hand gesture	HaGRID label	Class name	Output index	Game command
Closed fist	fist	left	0	LEFT
Open palm	palm	right	1	RIGHT
Thumbs up	like	up	2	UP
Thumbs down	dislike	down	3	DOWN

The mapping is saved in *dataset/class_mapping.json* and inside the model checkpoint itself. When the application loads the model, it compares the two and refuses to start if they differ, so output index 2, for example, can never quietly come to mean something other than UP.

6.3 System Architecture

Figure 1 shows how the pieces fit together. The application runs as two parts at the same time. A recognition worker thread grabs frames from the webcam, cuts out the hand region, runs the CNN and applies the confidence threshold and smoothing, while the main thread runs the game at 60 frames per second. The two sides share a single snapshot that always holds the latest recognition result and nothing older, so the game never has to work through a queue of stale predictions. Once per frame, a small gesture controller checks that snapshot. If it contains a fresh, stable command, the controller passes it to the game as a direction request. Key presses go through exactly the same request interface.

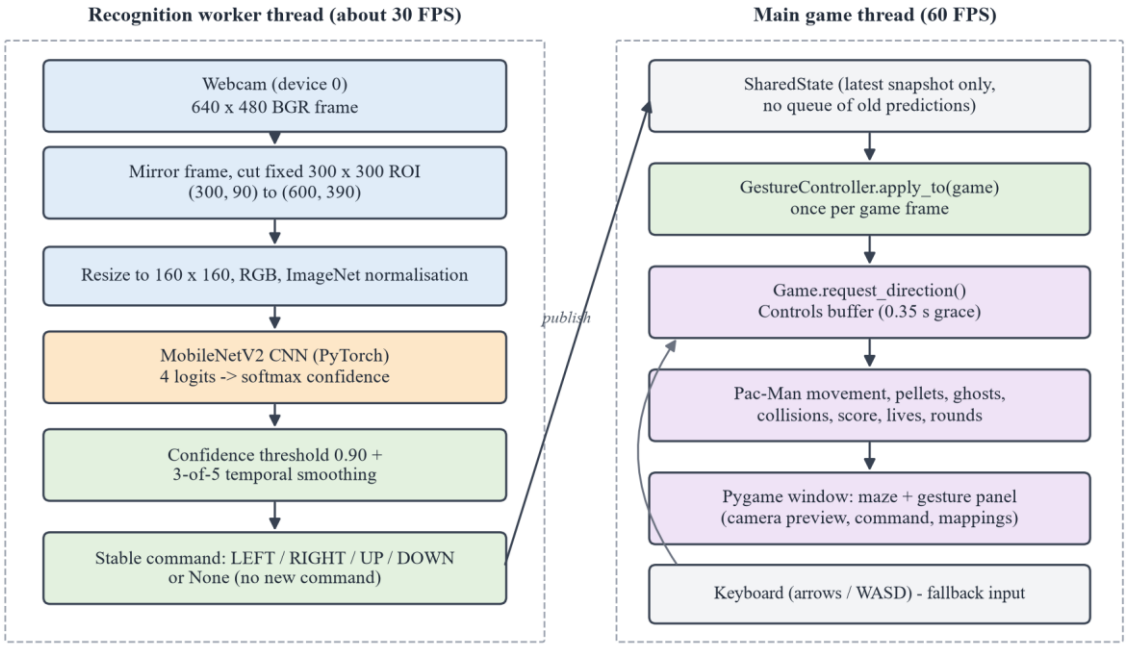


Figure 1: Proposed CNN-Based Gaming Application Architecture

7. DATASET DESCRIPTION

Our dataset was built from **HaGRID** (HAnd Gesture Recognition Image Dataset) by Kapitanov et al. [1], a large public collection of photographs of people making hand gestures in everyday indoor settings. It suited this project for two reasons. First, it is varied, with many different people, skin tones, hand shapes, backgrounds and lighting conditions. Second, every photograph comes with an official bounding box around the hand making the gesture. HaGRID also contains exactly the four gestures we needed, under the labels *fist*, *palm*, *like* (thumbs up) and *dislike* (thumbs down).

The images came from *cj-mills/hagrid-sample-500k-384p* on Hugging Face [2]. This is a repackaged version of HaGRID with 509,323 images scaled down to 384 pixels, together with the official annotation files. Because the images were scaled rather than cropped, HaGRID's bounding boxes still line up with them, which turned out to matter (Section 16). We did not download the whole 13.4 GB archive. Our script reads it over HTTP range requests and fetches only the annotation files and the images it actually uses. The source data is released under the Creative Commons Attribution-ShareAlike 4.0 licence (CC BY-SA 4.0) [14].

A HaGRID photograph shows a whole scene, often including the person's face and room, whereas the game only ever shows the network a tight square around the hand. To make the training images look like what the game sees, the script *src/crop_hagrid_hands.py* cut each photograph down to the annotated hand:

1. Take the largest bounding box with the required gesture label, skipping boxes smaller than 10% of the photograph's longer side.
2. Pad the box by 25% of its size on every side, so the crop keeps the whole hand, part of the wrist and some background, just like the webcam region.
3. Widen the shorter side to make the region square, shifting it rather than stretching it so that it stays inside the photograph.
4. Save the crop as an RGB JPEG image at quality 95, without resizing it.
5. Throw away any candidate whose crop is smaller than 96 pixels, whose photograph was already used for another class, or whose crop is byte-for-byte identical to one already accepted.

Candidates were processed in a fixed random order (seed 42), so running the script again produces exactly the same images. The thumbs-up and thumbs-down images come from

HaGRID's own *like* and *dislike* classes. We never made one from the other by flipping or rotating, because the two gestures differ only in which way the thumb points: a flipped thumbs-up is a thumbs-down with the wrong label. Each class ended up with 500 images, stored in *dataset/left*, *dataset/right*, *dataset/up* and *dataset/down*. Table 4 shows how they were divided, and Table 5 gives some properties measured from the image files.

Table 4: Dataset Distribution

Class (folder)	Gesture (HaGRID label)	Images	Training	Validation	Test
left	Closed fist (fist)	500	400	50	50
right	Open palm (palm)	500	400	50	50
up	Thumbs up (like)	500	400	50	50
down	Thumbs down (dislike)	500	400	50	50
Total		**2,000**	**1,600**	**200**	**200**

Table 5: Image Properties Measured from the Dataset Files

Class	Images	Shape	Side length range (pixels)	Average side (pixels)
left	500	Square, 3-channel RGB	96 to 384	138
right	500	Square, 3-channel RGB	96 to 362	152
up	500	Square, 3-channel RGB	96 to 384	143
down	500	Square, 3-channel RGB	96 to 384	147

Figure 2 shows five training images from each class. Even this small sample shows how much the images vary in background, lighting, skin tone, hand size and camera quality, and that variety is a large part of why the model copes with an unfamiliar webcam.



Figure 2: Sample Training Images from the Four Gesture Classes

One weakness of the split should be stated up front. It was made image by image, not person by person, and a later check (*reports/subject_leakage.json*) found that 82 of the 200 test images belong to people who also appear in the training set. To get a fairer measure of how the model handles new people, we built a separate evaluation set of 2,000 HaGRID hand crops (500 per class) from 1,727 people who appear nowhere in the main dataset. Its results are reported in Section 11.3. The dataset images themselves are not kept in the project's Git repository. They can be rebuilt from the source with *python src/crop_hagrid_hands.py --promote*, while the split file and class mapping are under version control.

8. DATA PREPROCESSING

Before an image reaches the network, it has to be brought to the fixed size and value range that the pretrained model expects. During training each image is also changed a little at random, so that the model learns the gesture itself rather than the details of particular photographs. All of this is done in *src/data_pipeline.py* with torchvision transforms, as listed in Table 6.

Table 6: Preprocessing Pipeline

Step	Training	Validation, test and live inference
1. Resize	160 × 160 pixels	160 × 160 pixels
2. Horizontal flip	Random, probability 0.5	None
3. Affine transform	Random rotation up to ±10°, translation up to 5%, scale 0.9 to 1.1	None
4. Colour jitter	Brightness and contrast varied by up to ±20%	None
5. Tensor conversion	RGB image to float tensor in [0, 1], shape (3, 160, 160)	Same
6. Normalisation	Mean [0.485, 0.456, 0.406], standard deviation [0.229, 0.224, 0.225]	Same

Resizing to 160 × 160 gives every image the same shape, and it is also the size stored in the checkpoint and used by the game. Normalising with the ImageNet channel means and standard deviations puts the pixel values in the same range as the images MobileNetV2 was originally trained on. Without this step, the pretrained features would be responding to inputs they were never tuned for.

The random changes, or augmentation, are applied to training images only. A horizontal flip turns a right hand into a left hand but does not change what the gesture means: a mirrored thumbs-up still points up. Small rotations, shifts and changes of scale imitate the way a player's hand never sits in exactly the same spot, and the brightness and contrast changes stand in for different rooms and webcams.

Two common augmentations were left out on purpose. A vertical flip would turn a thumbs-up into a thumbs-down while keeping the label "up", which would actively teach the model the wrong answer. Large rotations cause a milder version of the same problem, since a thumbs-up rotated far enough starts to look like a sideways thumb, so rotation is capped at 10 degrees. The project's pipeline check fails if a vertical flip ever finds its way into the training transforms.

Validation, test and live images go through a fixed transform with no randomness at all. This keeps the results repeatable and means the webcam image is processed in exactly the same way as the test images were. In the game, each 640 × 480 webcam frame is first flipped horizontally so that the preview behaves like a mirror. The fixed 300 × 300 hand region is then cut out, converted from OpenCV's BGR colour order to RGB and passed through the same transform.

To split the data, the sorted file list of each class was shuffled with seed 42 and divided into 400 training, 50 validation and 50 test images. That is an 80/10/10 split with every class equally represented. The result is saved in *dataset/data_splits.json* and was never regenerated after training. An automated test rebuilds the split and checks both that it matches the saved file exactly and that no image appears in more than one subset. The class indices are fixed as left = 0, right = 1, up = 2 and down = 3 instead of being taken from the order of the folders.

9. CNN MODEL DESIGN

Our classifier is **MobileNetV2** [3] from torchvision, loaded with weights pretrained on ImageNet (*MobileNet_V2_Weights.IMAGENET1K_V1*). We did not design the network from scratch. The convolutional feature extractor that MobileNetV2 learned on ImageNet is reused as it is, and only the final layer is replaced: in *src/train_model.py*, the original 1,000-class linear layer is swapped for a new one that maps 1,280 features to our 4 classes. The resulting model has **2,228,996 parameters** in total.

9.1 CNN Architecture

Table 7 walks through the network in the order it runs, with the output shape of each stage for one 160 × 160 image. We obtained these shapes by passing a test tensor through the actual model rather than copying them from a paper. Altogether, the feature extractor has 52 convolution layers, 52 batch-normalisation layers, 35 ReLU6 activations and 17 inverted residual blocks. In the table, *t* is the expansion factor of a block.

Table 7: MobileNetV2 Architecture as Implemented (output shapes for one 160 × 160 image)

No.	Layer / stage	Configuration	Output shape
1	Input	RGB image, normalised	3 × 160 × 160
2	features[0]	Convolution 3 × 3, 32 filters, stride 2, batch norm, ReLU6	32 × 80 × 80
3	features[1]	Inverted residual, t = 1, depthwise 3 × 3, projection to 16 channels	16 × 80 × 80
4	features[2-3]	2 inverted residual blocks, t = 6, 24 channels, first with stride 2	24 × 40 × 40
5	features[4-6]	3 inverted residual blocks, t = 6, 32 channels, first with stride 2	32 × 20 × 20
6	features[7-10]	4 inverted residual blocks, t = 6, 64 channels, first with stride 2	64 × 10 × 10
7	features[11-13]	3 inverted residual blocks, t = 6, 96 channels, stride 1	96 × 10 × 10
8	features[14-16]	3 inverted residual blocks, t = 6, 160 channels, first with stride 2	160 × 5 × 5
9	features[17]	Inverted residual block, t = 6, 320 channels	320 × 5 × 5
10	features[18]	Convolution 1 × 1, 1,280 filters, batch norm, ReLU6	1280 × 5 × 5
11	Pooling	Adaptive average pooling to 1 × 1	1280 × 1 × 1
12	Flatten	torch.flatten(x, 1)	1280

No.	Layer / stage	Configuration	Output shape
13	classifier[0]	Dropout, p = 0.2 (active only during training)	1280
14	classifier[1]	Fully connected layer 1,280 → 4 (5,124 parameters): output layer	4 logits
15	Softmax	Applied to the logits at inference to obtain confidences (not a model layer)	4 probabilities

9.2 Explanation of Layers

The assignment asks for a convolution layer, an activation function, pooling, flattening, a fully connected layer and an output layer. MobileNetV2 has all six. Some of them do not appear as separate lines in our own code because they live inside torchvision's forward pass, so the explanations below also say where each one actually happens.

Convolution Layer

A convolution layer slides small learnable filters across the image and produces feature maps that show where each pattern appears. Early layers pick up simple things such as edges and textures, while deeper layers respond to larger shapes such as fingers and the outline of a hand. MobileNetV2 uses three kinds of convolution. The very first layer is an ordinary 3 × 3 convolution with 32 filters and a stride of 2. After that, each inverted residual block works in three steps: a 1 × 1 *expansion* convolution multiplies the number of channels by t, a 3 × 3 *depthwise* convolution filters each channel on its own, and a 1 × 1 *projection* convolution brings the channel count back down. Splitting the work this way needs far fewer parameters and calculations than one full 3 × 3 convolution, and that is the main reason the model is quick enough for real-time use. When a block's input and output have the same shape, a shortcut adds the input straight to the output (a residual connection). Every convolution is followed by batch normalisation [15], which keeps training stable.

Activation Functions

The activation function in MobileNetV2 is **ReLU6**, f(x) = min(max(0, x), 6). It behaves like the familiar ReLU, letting positive values through and setting negative ones to zero, except that it also caps values at 6, which suits reduced-precision arithmetic. There are 35 of them: after the first convolution, after the expansion and depthwise convolutions of every block, and after the final 1 × 1 convolution. The projection convolution at the end of each block has no activation on purpose. The MobileNetV2 authors call this a *linear bottleneck*, because applying

ReLU to such a narrow representation would throw information away [3]. At the very end, softmax turns the four output scores into probabilities that add up to one.

Pooling

MobileNetV2 has no max-pooling layers. Instead, the feature maps shrink whenever a convolution uses a stride of 2, which halves their height and width each time (160 → 80 → 40 → 20 → 10 → 5). After the last convolution, **adaptive average pooling** takes each of the 1,280 feature maps, which are 5 × 5 at that point, and replaces it with its average value. The result describes how strongly each learned feature shows up anywhere in the image. Because the exact position of the hand in the box no longer matters, the classifier becomes more forgiving of where the player holds their hand, and the pooling adds no parameters. This step happens inside torchvision's MobileNetV2 forward method as *adaptive_avg_pool2d(x, (1, 1))*.

Flattening

After pooling, each image is represented by a 1280 × 1 × 1 block of numbers. The call *torch.flatten(x, 1)* in the same forward method reshapes this into a plain vector of 1,280 values, which is the form a fully connected layer expects.

Fully Connected (Dense) Layer

The classifier head is a dropout layer followed by a single fully connected (linear) layer. During training, dropout [16] randomly sets 20% of the feature values to zero, which stops the classifier from leaning too heavily on a handful of features and helps against overfitting. It is switched off during evaluation and gameplay. The linear layer multiplies the 1,280-value vector by a 4 × 1,280 weight matrix and adds 4 biases, which comes to 5,124 trainable parameters. It is the only completely new layer in the network: it replaced ImageNet's 1,000-class layer and started from random values.

Output Layer

The same linear layer is also the output layer. It produces four raw scores, or logits, always in the order left, right, up, down, which matches the class indices stored in the checkpoint. During training, the cross-entropy loss applies softmax internally. In the game, we apply softmax ourselves to get a confidence for each gesture, take the most likely one and accept it only if its

confidence is 0.90 or more (Section 12.3). There is no fifth "no gesture" output; a low confidence is simply treated as "no new command".

9.3 CNN Architecture Diagram

Figure 3 summarises the network, showing the output shape after each stage and which parts were frozen or trained in the two training stages described in Section 10.

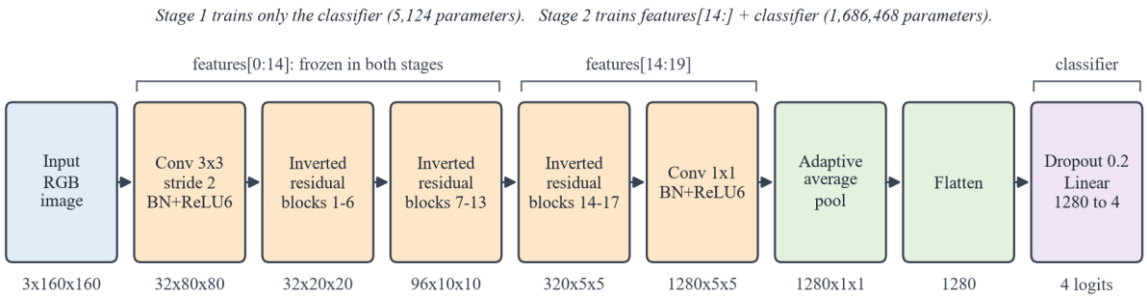


Figure 3: MobileNetV2 Architecture Adapted for Four Gesture Classes

10. MODEL TRAINING

Training was done with *src/train_model.py* on the 1,600 training images, while the 200 validation images were used to follow progress and to pick the best checkpoint. The script never loads the test images at all, and the saved training history records this as *test_split_used: false*. Table 8 lists the settings that were actually used.

Table 8: Training Configuration

No.	Parameter	Value used
1	Framework	PyTorch 2.14.0 with torchvision 0.29.0
2	Base model	MobileNetV2, ImageNet weights IMAGENET1K_V1, new 4-class output layer
3	Input size	3 × 160 × 160 (RGB)
4	Dataset split	1,600 training / 200 validation / 200 test (80% / 10% / 10%), stratified, seed 42
5	Epochs	13 in total: 5 in stage 1 and 8 in stage 2
6	Stage 1 (classifier training)	Feature extractor frozen; 5,124 trainable parameters; learning rate 0.001
7	Stage 2 (fine-tuning)	features[14:] and classifier trainable; 1,686,468 trainable and 542,528 frozen parameters; learning rate 0.0001
8	Batch size	32

No.	Parameter	Value used
9	Optimiser	AdamW [9], weight decay 0.0001, a new optimiser for each stage; constant learning rate (no scheduler)
10	Loss function	Cross-entropy loss
11	Activation functions	ReLU6 in the feature extractor; softmax applied to the outputs for confidence
12	Regularisation	Data augmentation (Table 6), dropout 0.2, weight decay
13	Early stopping	Stage 2 only, patience 3 epochs on validation loss; not triggered
14	Model selection	Lowest validation loss, validation accuracy as tie-break
15	Mixed precision	CUDA automatic mixed precision: float16 autocast with gradient scaling [10]
16	Hardware and time	RTX 3050 Ti Laptop GPU; 62.96 s in total; peak GPU memory 161.1 MB

Stage 1 trained only the new classifier. Every parameter in the feature extractor was frozen, leaving just the 5,124 parameters of the new 1,280 → 4 layer to learn, for 5 epochs at a learning rate of 0.001. The idea is to let the new layer find a reasonable mapping from the existing ImageNet features to our four gestures before any pretrained weights are touched. A freshly initialised layer produces large, almost random gradients at first, and letting those flow back into the pretrained layers could damage what they have already learned. Over this stage, validation accuracy went from 84.0% to 88.5%, with a peak of 90.5% in the fourth epoch.

Stage 2 fine-tuned the end of the network. The last five feature blocks (*features[14:]*, the final inverted residual blocks and the 1 × 1 convolution) were unfrozen along with the classifier, giving 1,686,468 trainable parameters, while the earlier 542,528 stayed frozen. The learning rate was cut tenfold to 0.0001 so that the pretrained weights would shift gently towards hand-specific features. The early layers detect general edges and textures that are useful for any image, and with only 1,600 training images it made sense to leave them alone. Early stopping with a patience of 3 epochs was switched on but never triggered, so all 8 planned epochs ran.

We used mixed-precision training, in which most calculations run in 16-bit floating point while a gradient scaler stops very small gradients from rounding to zero. This saves memory and speeds training up on the GPU. The whole run took 62.96 seconds. The first epoch alone took 22.06 seconds, most likely because it included one-time start-up work such as initialising the GPU; after that, each epoch took between about 3 and 5.4 seconds. At the end of every epoch the script checked the validation set and saved the model whenever the validation loss improved. The best checkpoint came from the very last epoch (stage 2, epoch 8) and is stored

as *model/best_direction_model.pt* (9,165,963 bytes), together with its class mapping, input size, normalisation values and training details.

11. MODEL PERFORMANCE EVALUATION

Every number in this section comes from files the project saved at the time: the epoch-by-epoch training history in *model/direction_training_history.json*, and the test evaluation in *reports/p5_evaluation/*, which was run once after training with the model in evaluation mode and 32-bit precision. Nothing was re-measured or adjusted for this report. Table 9 gives the full training history.

Table 9: Epoch-wise Training History (from model/direction_training_history.json)

Epoch	Stage	Train loss	Train acc. (%)	Val. loss	Val. acc. (%)
1	1	0.9223	67.00	0.6270	84.0
2	1	0.5337	83.00	0.4829	86.5
3	1	0.4313	86.25	0.4174	88.0
4	1	0.3730	88.38	0.3554	90.5
5	1	0.3520	88.69	0.3509	88.5
6	2	0.1797	93.50	0.1090	97.5
7	2	0.0507	98.38	0.0617	98.5
8	2	0.0449	98.44	0.0726	98.0
9	2	0.0280	99.38	0.0499	98.0
10	2	0.0320	99.06	0.0398	98.5
11	2	0.0203	99.38	0.0567	98.0
12	2	0.0104	99.75	0.0461	97.5
13	2	0.0083	99.81	0.0362	99.5

11.1 Training Accuracy

Training accuracy is measured on the augmented training images as each epoch runs. While only the new classifier was learning in stage 1, it climbed from 67.0% to 88.7% and then levelled off: frozen ImageNet features on their own could not fully separate gestures that differ mainly in the direction of the thumb. Once the last feature blocks were unfrozen, it improved quickly, reaching 93.5% after the first fine-tuning epoch and **99.81%** by the last one, while the training loss dropped from 0.922 to 0.0083.

11.2 Validation Accuracy

Validation accuracy followed a similar path: 84.0% to 88.5% in stage 1, then 97.5% after the first fine-tuning epoch. For the next six epochs it moved between 97.5% and 98.5%, which on

200 images is a difference of only two or three pictures. In the final epoch it reached **99.5%** (199 of 200), and the validation loss was at its lowest there too (0.0362), so that epoch's weights became the final model. The one validation image it got wrong was a fist (*left*) predicted as *down*. Not a single thumbs-up was mistaken for a thumbs-down, or the other way round.

11.3 Test Accuracy

The 200 test images were used exactly once, after training had finished. The model got 198 of them right, **an accuracy of 99.00%**, with a test loss of 0.0292 (0.029165). With only 200 images, a single mistake moves the accuracy by half a percentage point, so we also calculated a 95% Wilson confidence interval [17], which runs from 96.4% to 99.7%. Table 10 shows the precision, recall and F1-score for each class, taken from the saved evaluation file.

Table 10: Per-class Test Metrics (from direction_evaluation_metrics.json)

Class	Precision	Recall	F1-score	Support
left (fist)	1.0000	1.0000	1.0000	50
right (palm)	1.0000	0.9800	0.9899	50
up (like)	0.9804	1.0000	0.9901	50
down (dislike)	0.9800	0.9800	0.9800	50
Macro average	0.9901	0.9900	0.9900	200
Weighted average	0.9901	0.9900	0.9900	200

Section 7 mentioned that 82 of the test images come from people who are also in the training set, which could flatter the test result. The separate set of 2,000 images from 1,727 people who appear nowhere in the dataset deals with that concern. The model classified 1,992 of them correctly (99.6%, with a 95% interval of 99.2% to 99.8%) and reached a macro F1-score of 0.996. That is slightly higher than the test accuracy, although the difference is not statistically significant. Table 11 puts the main results side by side.

Table 11: Summary of Evaluation Results

Evaluation	Images	Accuracy	Other measures
Validation set (model selection)	200	99.5% (199/200)	Validation loss 0.0362
Test set (held out, used once)	200	99.0% (198/200)	Test loss 0.0292; macro F1 0.9900; weighted F1 0.9900
Unseen-people set	2,000	99.6% (1,992/2,000)	Macro F1 0.996; 1,727 people not in the dataset

On the development GPU, a single forward pass on one image took 6.0 ms on average in the evaluation script (median 6.1 ms, 95th percentile 6.9 ms over 300 runs). This was measured on its own, without the camera or the game running.

11.4 Training and Validation Accuracy Graph

Figure 4 plots the training and validation accuracy for every epoch in Table 9, with a dashed line where fine-tuning starts. The jump right after that line is the clearest sign that adapting the last feature blocks was worthwhile. The two curves stay close together throughout. In stage 1 the validation accuracy is actually the higher of the two, probably because augmentation makes the training images harder than the clean validation images. By the end the gap is small, so there is no sign of serious overfitting.

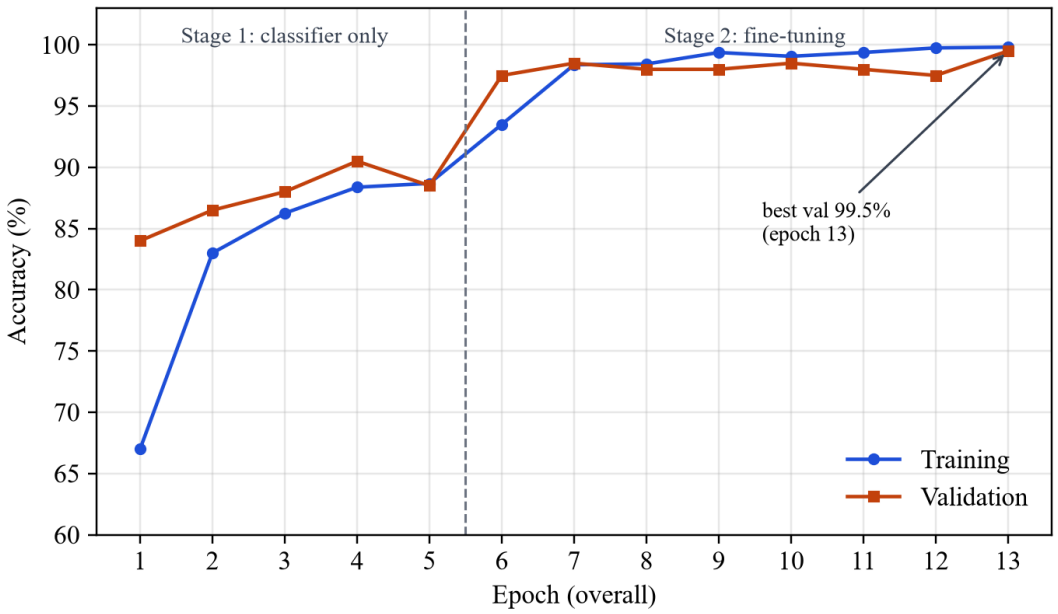


Figure 4: Training and Validation Accuracy per Epoch

11.5 Training and Validation Loss Graph

Figure 5 shows the matching loss curves. Both fall steadily in stage 1 and drop sharply once fine-tuning begins. In stage 2 the training loss keeps going down, while the validation loss flattens out somewhere between 0.04 and 0.07 before reaching its lowest point, 0.0362, in the final epoch. That is the epoch whose weights were saved.

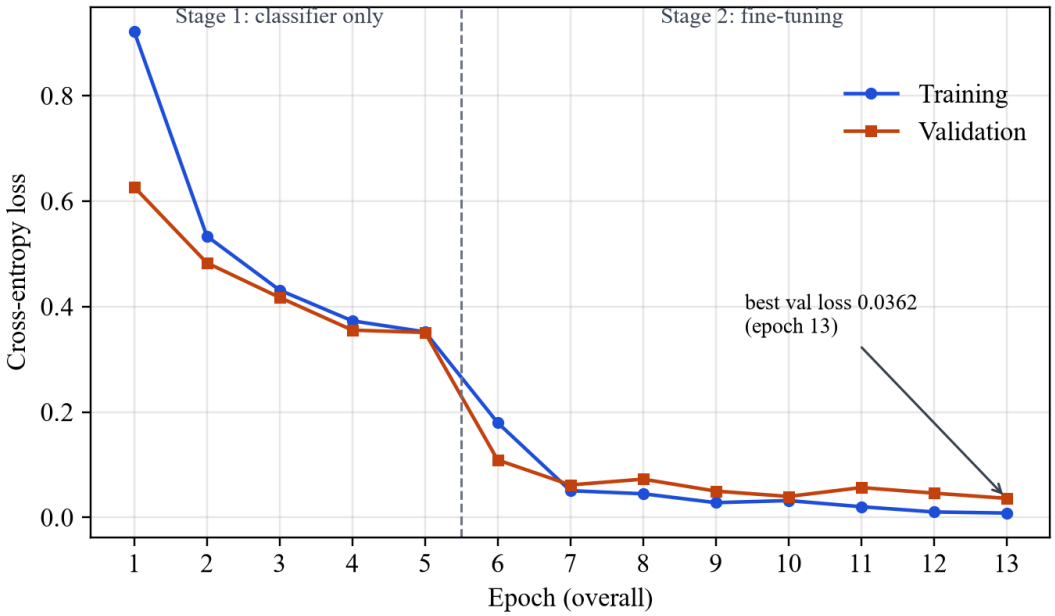


Figure 5: Training and Validation Loss per Epoch

11.6 Confusion Matrix

Figure 6 is the confusion matrix for the test set, built from the saved evaluation results. Each row is a true class and each column a predicted class. All 50 fists and all 50 thumbs-ups were classified correctly. There were two mistakes: one open palm was called a thumbs-down with a confidence of only 0.603, and one thumbs-down was called a thumbs-up with a confidence of 0.903.

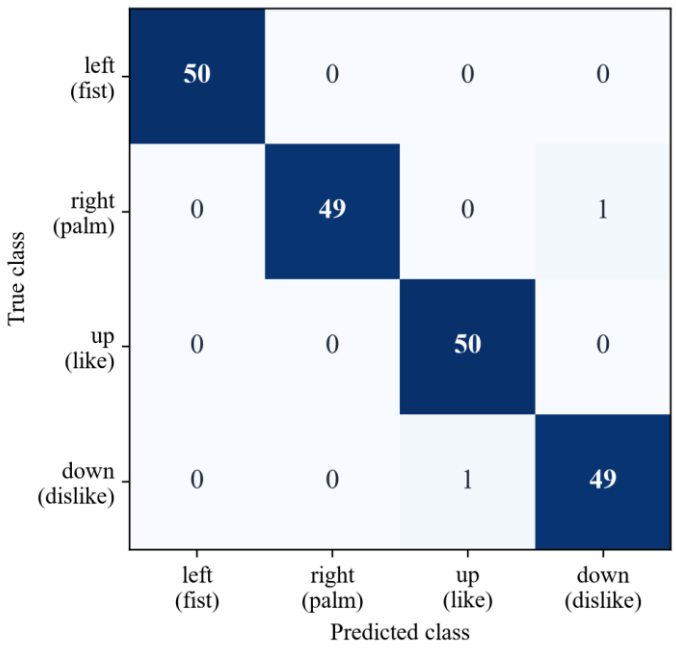


Figure 6: Confusion Matrix on the 200-Image Test Set

In the game, these two mistakes would play out differently. The first falls below the 0.90 threshold, so it would produce no command at all rather than a wrong one. The second just clears the threshold, but a single frame cannot turn Pac-Man, because a command only becomes stable once the same gesture wins three of the last five frames. Across the 198 correct predictions, the lowest confidence was 0.618.

11.7 Sample Predictions

Figure 7 shows eight real test images with their true labels and the predictions saved in *reports/p5_evaluation/direction_test_predictions.csv*. The top row has the most confident correct prediction for each class. The bottom row has the two correct predictions the model was least sure about, followed by the two images it got wrong, outlined in red. The harder images tend to be less typical examples of their class; the misclassified thumbs-down, for instance, is noticeably blurred.

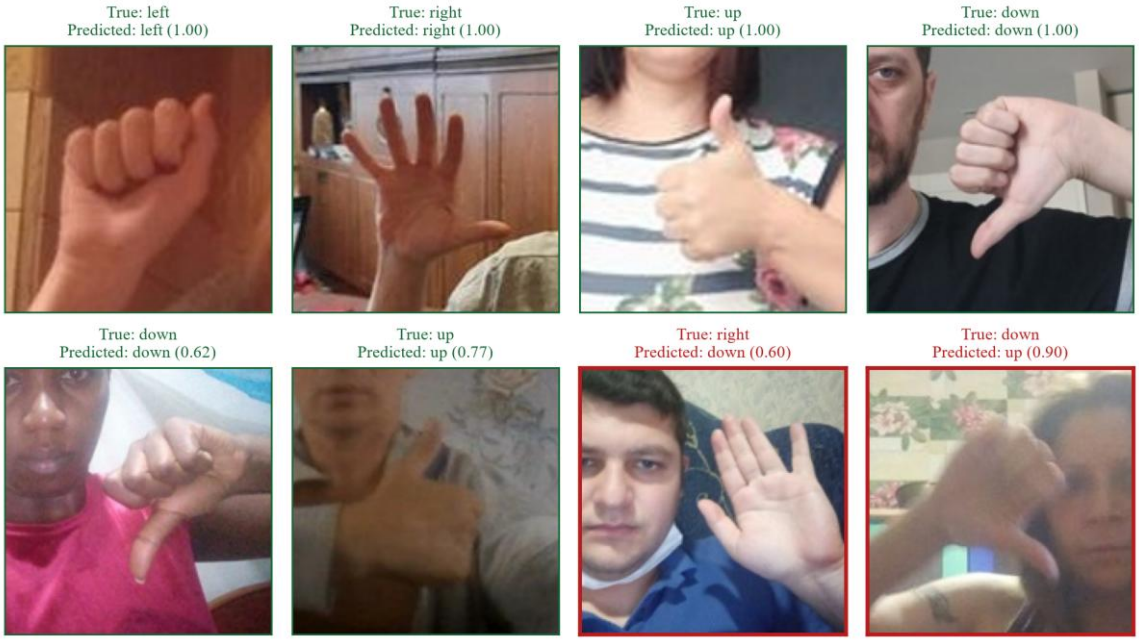


Figure 7: Sample Test Predictions (green: correct, red: misclassified; value in brackets: confidence)

12. GAME DEVELOPMENT

12.1 Game Interface

When the application starts, it shows the start screen in Figure 8, which explains the four gestures and the keyboard keys while the model loads in the background. A status line shows what is happening (for example, "Initializing gesture recognition..."), and the game begins when the player presses SPACE or ENTER. The recognizer is created only once, at this point, so there is no loading pause later on, not even after a restart.

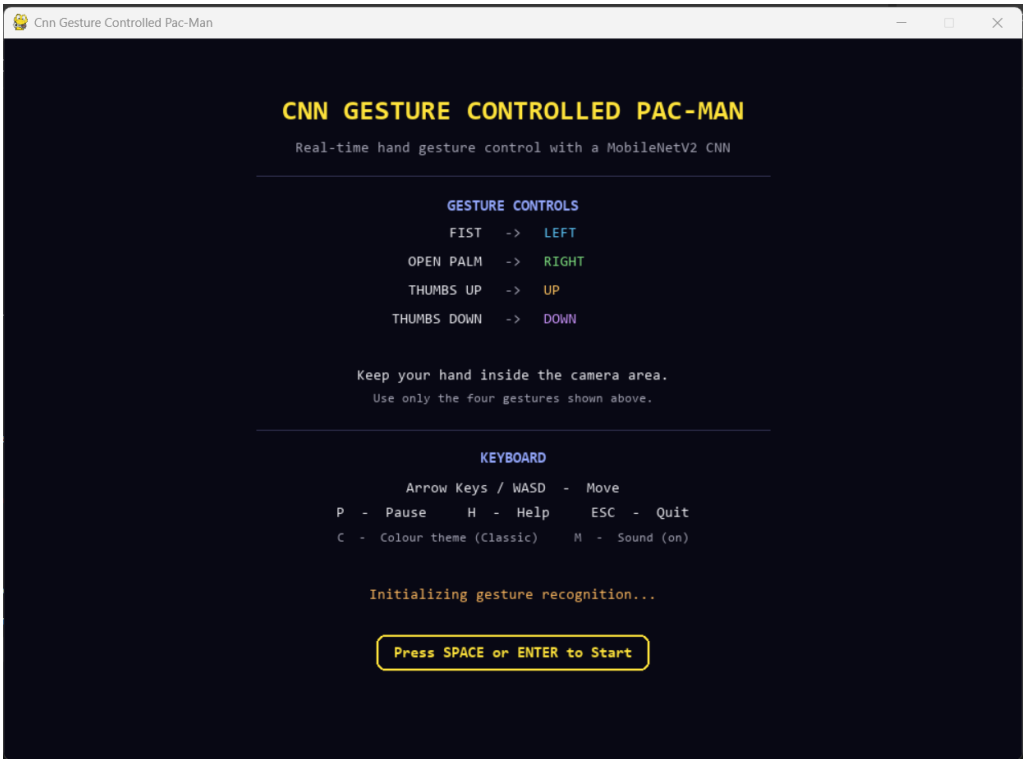


Figure 8: Start Screen of the Application

Figure 9 shows the game in the middle of gesture play. On the left is the maze, with the score, high score and round number above it and the remaining lives and current gesture below it. The gesture panel on the right contains, from top to bottom:

- a large 376 × 376-pixel preview of the camera region, exactly as the CNN receives it, with a reminder to keep the hand inside the box;
- the current gesture command (RIGHT in the screenshot) and its confidence (99.9%);
- the gesture-control table, with the active gesture and direction highlighted;
- a status line for gesture control, such as READY, or a message when the camera has no picture;

- the keyboard keys, and a small diagnostic line with the raw prediction, the inference time, the device in use (GPU or CPU) and the frame rate.

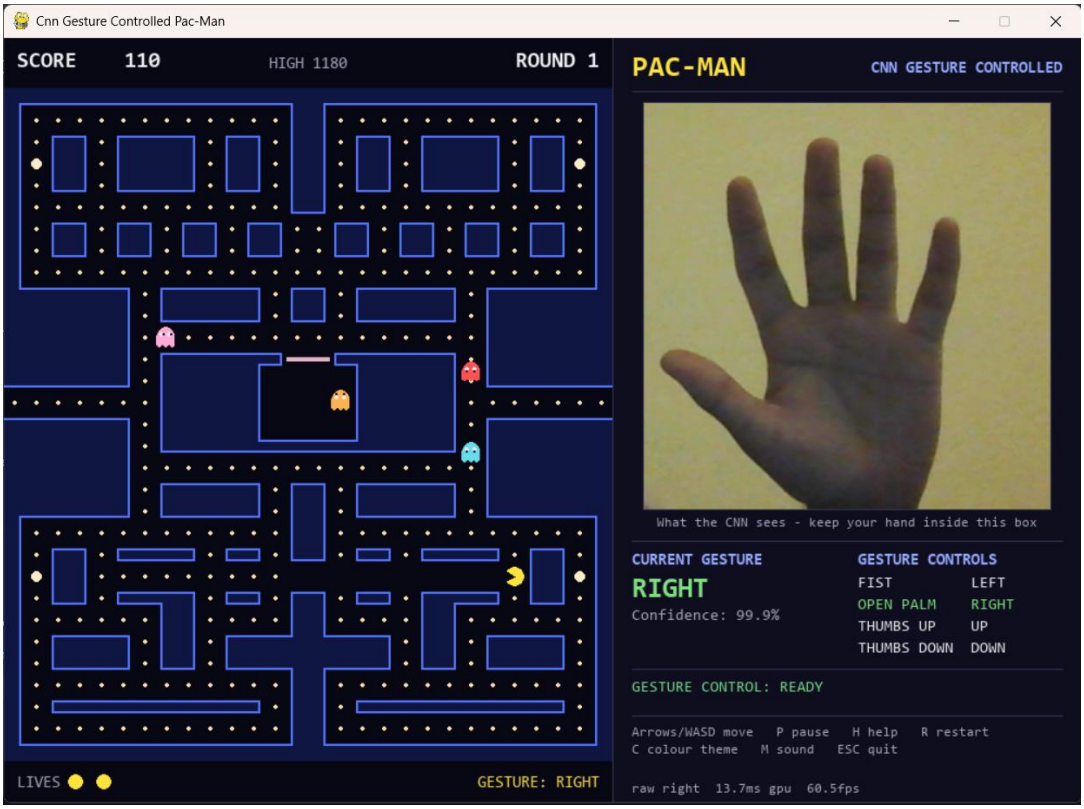


Figure 9: Gameplay with the Live Camera View and Gesture Panel

Short banners explain every pause: "READY!" before play starts or resumes, "LIFE LOST" with the number of lives left, "ROUND N CLEARED!", "PAUSED", and a game-over panel showing the final score, the round reached and the keys to restart or quit. The player can also pause (P), open a help screen (H), switch the colour theme (C), mute the sound (M) or quit (ESC).

12.2 Game Logic

The main loop runs 60 times a second. On each pass it handles keyboard and window events, lets the gesture controller hand over the latest stable command, moves the game forward by the time that has passed (capped at 0.05 s, so that an occasional slow frame cannot cause a big jump), and then draws the maze, the characters and the panel.

The maze is a grid of 20 × 20-pixel tiles, and Pac-Man moves at 6.2 tiles per second. Whether it comes from a gesture or a key, every input becomes a *direction request* that stays valid for 0.35 seconds. Reversing direction happens straight away; any other turn is taken at the first tile centre where it is possible. This buffer lets the player make a gesture slightly before reaching

a junction, which makes up for the time it takes to recognise one. Row 14 is a tunnel, so anything that leaves the maze on one side reappears on the other.

Pac-Man eats a pellet by reaching its tile, and he collides with a ghost when their centres come within 0.6 tiles of each other. Table 12 lists the scoring rules and the other key parameters set in *game/engine.py*.

Table 12: Scoring Rules and Game Parameters

Item	Value
Normal pellet / power pellet	10 points (330 pellets) / 50 points (4 power pellets)
Eating frightened ghosts in sequence	200, 400, 800 and 1,600 points; the sequence restarts with each power pellet
Bonus fruit	Appears after 70 and 170 pellets in a round for 9.5 s; 100 points in round 1, rising to 5,000
Lives	3 at the start; one extra life once, at 10,000 points
Frightened time	6.0 s in round 1, 0.5 s shorter each round, at least 2.0 s; ghosts flash in the last 2 s
Ghost speed	5.4 tiles/s, +0.2 per round, at most 6.0 tiles/s (always slower than Pac-Man)
High score	Saved between sessions; a tie is not counted as a new high score

Each ghost is always in one of five states: *house* (waiting in the ghost house in the middle of the maze), *scatter* (heading for its own corner), *chase* (going after a target tile), *frightened* (slower and wandering at random after a power pellet) or *eaten* (racing back to the house as a pair of eyes). The ghosts leave the house 0, 2, 6 and 10 seconds into a round. They switch between scatter and chase on a fixed timetable of 7 s scatter, 20 s chase, 7 s scatter, 20 s chase, 5 s scatter, 20 s chase and 5 s scatter, and then chase for the rest of the round. At every tile, a ghost picks the move that brings it closest to its target. The only real difference between the four ghosts is which tile they aim for while chasing, as Table 13 shows.

Table 13: Ghost Roles and Chase Targets

Ghost role	Chase target
Chaser	Pac-Man's current tile. It also speeds up when 20 or fewer pellets remain in the round.
Ambusher	The tile four tiles ahead of Pac-Man in his direction of travel
Flanker	The Chaser's position reflected through the point two tiles ahead of Pac-Man, so that it approaches from the other side

Ghost role	Chase target
Drifter	Pac-Man's tile when more than eight tiles away; otherwise it retreats to its scatter corner

Figure 10 shows how the game moves between its states. If a ghost in scatter or chase mode touches Pac-Man, he loses a life and a short death sequence plays. If he has lives left, everyone returns to their starting positions, but the eaten pellets and the score stay as they were; if not, the game is over and the high score is updated. Eating the last pellet ends the round on the spot, even if a ghost reaches Pac-Man in the same frame, and the next round starts with a full board and harder ghosts. After a game over, pressing R starts a brand-new game. The recognizer's smoothing is reset whenever a life is lost, a round changes or the game restarts, so a gesture the player was holding during the pause cannot steer Pac-Man the moment he reappears.

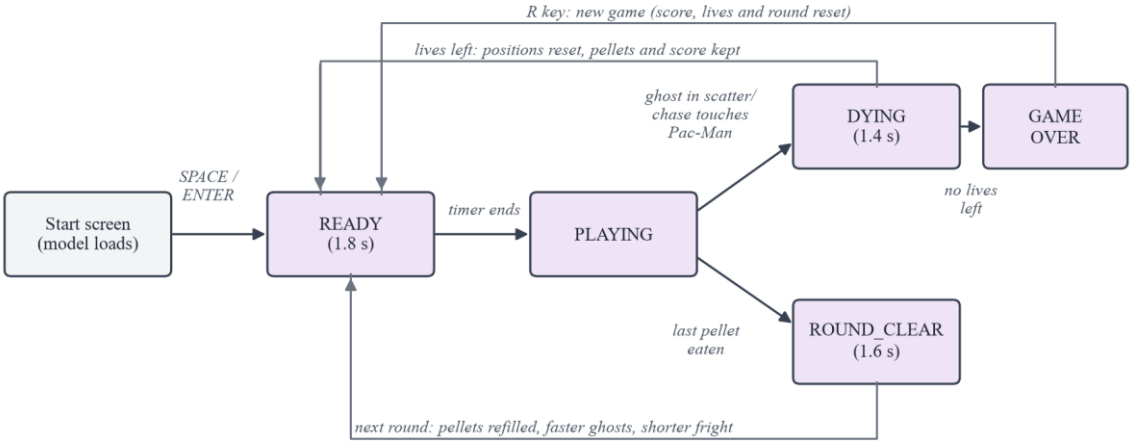


Figure 10: Game State Transitions

12.3 CNN-Game Integration

The integration code lives in three files: *src/gesture_recognizer.py* (the CNN and its decision logic), *src/game_integration.py* (the worker thread and the controller) and *src/play_gesture.py* (the application itself). Following the code, this is what happens to each frame:

1. The recognition worker thread reads a 640 × 480 frame from webcam device 0 with OpenCV.
2. The frame is mirrored horizontally, and the fixed 300 × 300 region from (300, 90) to (600, 390) is cut out. Frames that are completely dark or contain only static noise, which is what a webcam sends when its privacy switch or cover is closed, are recognised and never passed to the CNN.

3. The region is converted to RGB, resized to 160 × 160 and normalised with the ImageNet values, using the same fixed transform as the test set.
4. The trained MobileNetV2 produces four logits, softmax turns them into confidences, and the most likely gesture is picked.
5. If its confidence is below 0.90, the frame counts as "no gesture". The remaining predictions go into a five-frame window, and a gesture becomes the stable command only once it wins at least three of the last five frames.
6. The stable gesture is mapped to its direction (Table 3) and written to a shared snapshot that only ever holds the newest result.
7. Once per game frame, the gesture controller reads the snapshot. If it is fresh (less than 0.75 s old) and contains a stable command, the controller calls the game's *request_direction* method, the same one the keyboard uses. Otherwise it sends nothing.
8. The game updates movement, pellets, ghosts, collisions, score and state as described in Section 12.2.
9. The panel shows the camera preview, the current stable command and its confidence, the highlighted mapping and the status of gesture control.

A *None* result means "no new command", not "stop". Pac-Man carries on in the direction he was already going, and whatever request was pending simply runs out after 0.35 s. While the player keeps holding a gesture, the controller resends the request every frame, so the turn happens as soon as the maze allows it. Table 14 lists the recognition settings.

Table 14: Real-time Recognition Parameters

Parameter	Value
Camera	Device 0, 640 × 480 frames, mirrored
Region of interest	300 × 300 pixels, from (300, 90) to (600, 390)
Model input	3 × 160 × 160, ImageNet normalisation
Confidence threshold	0.90
Temporal smoothing	Window of 5 predictions, at least 3 in agreement
Snapshot freshness	Results older than 0.75 s are ignored
Direction request buffer	0.35 s
Device	CUDA GPU when available, otherwise CPU; the device is logged and shown in the panel
Camera failure handling	After 30 consecutive failed reads, up to 3 reconnection attempts 1 s apart; then CAMERA ERROR is shown and the keyboard keeps working

The 0.90 threshold and the 3-of-5 smoothing were chosen from live webcam measurements, never by tuning on the test set. In those recordings, deliberate gestures never dropped below 0.961 confidence, while an empty box or a relaxed hand never went above 0.617, so 0.90 sits comfortably in the gap. The recognizer uses a CUDA GPU if one is present. If not, it runs on the CPU and says so at start-up and in the panel instead of switching silently. The worker never holds up the game loop: all the game does is read the latest snapshot, which takes next to no time, and the worker overwrites that snapshot rather than queuing results.

13. IMPLEMENTATION

The project is written in Python and split into two packages with a firm boundary between them. *game/* holds the game and never imports PyTorch, OpenCV or any of the recognition code, while *src/* holds the dataset, training, evaluation and recognition code and puts the final application together. Keeping the two apart means the game can be run and tested without the CNN at all, and an automated self-test fails if a game module ever starts importing the recognition stack. Table 15 lists the main modules; the full source code is submitted separately.

Table 15: Major Modules of the Implementation

Module	Responsibility
game/engine.py	Game loop, states, scoring, lives, rounds, collisions, drawing and keyboard handling
game/maze.py, player.py, ghost.py, entity.py	Maze layout and tunnel, Pac-Man movement, ghost states and targeting, shared movement code
game/controls.py	Direction-request buffer shared by keyboard and gesture input
game/theme.py, audio.py, profile.py	Colour themes, synthesised sound effects, saved high score and settings
src/crop_hagrid_hands.py	Builds the 2,000-image dataset from HaGRID using the official bounding boxes
src/data_pipeline.py	Class mapping, deterministic split, preprocessing and augmentation, data loaders
src/train_model.py	Model construction, two-stage training, checkpoint saving and guarded loading
src/evaluate_model.py	One-time test-set evaluation, metrics, confusion matrix and prediction records
src/gesture_recognizer.py	Loads the checkpoint, preprocesses the region of interest, applies threshold and smoothing

Module	Responsibility
src/game_integration.py	Recognition worker thread, shared snapshot, gesture controller, camera error handling
src/play_gesture.py	The complete application: start screen, gesture panel, main loop, command-line options
tests/ and src/check_*.py	Automated unit and integration tests and standalone self-test scripts

Running *python src/play_gesture.py* creates the game and the gesture controller and starts the recognition worker. The worker loads the model, checking the checkpoint's SHA-256 checksum, loading it without running any code stored inside it and confirming its architecture, task, input size and class mapping, and then opens the camera. The start screen stays up until the player presses SPACE or ENTER. From then on the main loop runs until the player presses ESC or closes the window, at which point the worker is stopped and the camera is released.

Setting the project up takes one command, *python install.py*, which creates a virtual environment and installs the pinned dependencies. After that, the game can be started in any of the following ways. On Windows, double-clicking *setup.bat* once and then *run_game.bat* does the same thing.

python src/play_gesture.py starts the game with gestures and keyboard.

python src/play_gesture.py --no-camera starts it with the keyboard only, without loading the model.

python src/play_gesture.py --device cpu runs the model on the CPU even when a GPU is present.

To rebuild the dataset, retrain the model or repeat the evaluation, use *python src/crop_hagrid_hands.py --promote*, *python src/train_model.py* and *python src/evaluate_model.py*. The last two need an NVIDIA CUDA GPU.

14. RESULTS AND DISCUSSION

On the held-out test images, the trained MobileNetV2 was right 99.0% of the time, with precision and recall balanced across all four classes (Table 10) and only two mistakes (Figure 6). It scored 99.6% on 2,000 images of people it had never seen, so the test result cannot be put down to people shared between the training and test sets. The training curves (Figures 4 and 5) show only a small gap between training and validation, which suggests that transfer learning,

augmentation, dropout and choosing the checkpoint by validation loss together kept overfitting in check, even on a fairly small dataset.

We also wanted to know how the model holds up when the picture is worse than in the dataset. This was tested on the unseen-people images under simulated camera conditions (*reports/robustness.json*), and Table 16 shows a selection of the results. Moderate changes in lighting, colour, image quality and hand angle hardly matter. Strong blur and heavy sensor noise, which are what fast hand movement and a dark room produce, are a different story and cut accuracy sharply. That is why players are advised to play in a lit room and to hold each gesture for a moment.

Table 16: Accuracy under Simulated Camera Conditions (2,000 unseen-people images)

Condition	Accuracy
Original images	99.6%
Dim light (brightness × 0.4) / dark room (× 0.25)	99.4% / 98.1%
Harsh light (brightness × 1.6)	94.0%
Warm or cool colour cast	99.1% / 99.0%
Heavy JPEG compression (quality 15) / low resolution (48 px)	98.4% / 97.9%
Hand tilted by +20° / -20°	98.6% / 98.7%
Slight blur / strong blur	98.4% / 76.7%
Sensor noise (σ = 15) / heavy sensor noise (σ = 30)	94.2% / 58.6%

To see how the whole application behaves over time, it was run continuously for 330 seconds with the real webcam and the GPU model. Table 17 summarises that run, as recorded in the project's final test report, and every figure in the table comes from the same run. The game stayed at its 60 frames-per-second target, recognition kept up with the camera at about 30 frames per second, and no backlog ever built up: throughout the run, snapshot writes exceeded processed frames by exactly three, the three status messages sent at start-up. In four separate start-and-quit runs, the application was ready 1.3 to 2.3 seconds after launch and shut down in about a third of a second, with the camera released and no threads left behind.

Table 17: Real-time Performance of the Complete Application (330-second run)

Measure	Result
Game frame rate	60.1 FPS (95th-percentile frame time 17.0 ms)
Camera recognition rate	30.4 FPS

Measure	Result
CNN inference time in the application	Mean 18.26 ms, median 17.12 ms, 95th percentile 27.40 ms
Average age of the snapshot read by the game	About 17 ms
Backlog of old predictions	None (snapshot writes minus frames constant at 3)
Restarts after game over during the run	6, without reloading the model or reopening the camera

Inside the application, inference took longer than the 6.0 ms measured on its own in Section 11.3. The in-game timing also covers preparing each image and copying it to the GPU, and the camera, the recognizer and the game all share the same laptop. Even so, there is plenty of room left within the 33 ms available for each camera frame. Over a ten-minute simulated session the game's memory use did not grow, and the model never used more than 22.2 MB of GPU memory.

Live trials, with a person in front of the webcam, were recorded in *reports/p6_live/* and are summarised in Table 18. Every held gesture produced the right command, and not one trial without a gesture produced a command. Switching from one gesture to another took a median of 1.26 s in the final run. Most of that is simply the time it takes to move a hand: once the model first saw the new gesture, it took 143 ms on average to turn it into the stable command.

Table 18: Live Webcam Trials (one webcam, one room)

Trial	Result
Held gestures, 20 per class	80 of 80 correct stable commands; lowest confidence 0.961
No hand (20) and relaxed hand (20)	40 of 40 produced no command; highest confidence 0.617
Gesture changes, final run	6 changes, none passed through a wrong command; median 1.26 s
Gesture changes, first attempt	12 changes, 2 passed briefly through a wrong command; median 2.90 s

Table 19 lists the self-test suites recorded in the project's final test report. On top of these, the project has a pytest suite of 546 automated tests covering the data pipeline, the model-loading checks, the recognition logic, the game rules, the integration and the user interface, which runs in continuous integration on Windows, macOS and Linux. Passing tests show that the program

does what it should in the situations that were tested. They cannot prove that there are no bugs at all, which is why live trials and manual play were also part of the testing.

Table 19: Recorded Automated Self-test Results

Suite	What it verifies	Result
src/realtime_gesture.py --selftest	Recognizer with the live webcam and CUDA	15 / 15 passed
game/main.py --selftest	Game rules and mechanics	58 / 58 passed
src/check_integration.py	CNN-game integration	28 / 28 passed
src/check_ui.py	User interface	21 / 21 passed
src/check_final_application.py	Contracts between all subsystems	27 / 27 passed
Total		**149 / 149 passed**

Finally, the game was played by hand. We checked the gesture directions, turning early before junctions, the tunnel, ghost movement, power pellets, losing a life, restarting and whether the recognition delay felt acceptable in practice, and all of these were accepted on 13 September 2026.

Looking back at Section 3.3, all seven objectives were met. The dataset was prepared and documented (Section 7), a MobileNetV2 transfer-learning model was designed and trained (Sections 9 and 10) and then evaluated on an untouched test set and on unseen people (Section 11), an original maze game was built (Section 12), the CNN was connected to it in real time with the keyboard as a backup (Section 12.3), and the result was tested both automatically and live (this section).

16. CHALLENGES FACED

The first problem was the difference between the dataset and the webcam. HaGRID photographs show whole scenes, but the game only shows the network a small square around the hand, so we cropped every image using the official bounding boxes. Our first attempt used a different repackaging of HaGRID, and the crops came out wrong. Drawing the boxes on top of the images showed why: that version's images had been cropped rather than resized, so the official boxes no longer matched the pixels. Switching to the downscaled 384-pixel sample fixed it.

Thumbs-up and thumbs-down caused a less obvious problem. They differ only in which way the thumb points, so standard augmentations such as vertical flips or large rotations would have produced images with the wrong label. We restricted augmentation to horizontal flips and rotations of at most 10 degrees, and added an automated check so that this cannot slip back in by accident.

Deciding when to trust a prediction took some thought. Acting on every single frame would let one uncertain frame send Pac-Man the wrong way. The 0.90 threshold and 3-of-5 smoothing came from live measurements, which showed a wide gap between real gestures (never below 0.961 confidence) and idle hands (never above 0.617).

Timing was another issue. Recognising a gesture reliably takes a fraction of a second, which is longer than Pac-Man spends at a junction. A 0.35-second direction buffer solved this. We first tried 0.30 seconds, but in testing it missed a turn two tiles away by a few hundredths of a second.

Running the CNN inside the game loop would have tied the game's frame rate to the camera, so recognition was moved to its own thread. The thread publishes only its latest result, which means the game never waits for the camera and never acts on stale predictions.

One risk was easy to miss: loading the wrong model. A checkpoint from an earlier version of the project had the same four outputs but different meanings for them, so it would have loaded without any error and then steered the game wrongly. The loader now checks the checkpoint's checksum and its stored class mapping instead of trusting the output shape.

Finally, the development laptop has a hardware privacy switch for the webcam. With the switch off, the camera stays open but sends black or noisy frames, and the CNN would happily classify those. The worker now recognises such frames, skips them and tells the player what to check.

17. LIMITATIONS

- The model only knows four gestures and has no "none of these" class. A clear gesture it was never trained on, such as a peace sign or three raised fingers, can be read confidently as one of the four; in live testing, a peace sign came out as RIGHT with 99.3% confidence. Relaxed or absent hands are not a problem, because the confidence threshold filters them out.
- Low light and fast movement hurt accuracy. Heavy camera noise in a dark room and strong motion blur both reduce it considerably (Table 16).
- The hand has to be inside a fixed 300 × 300 box on the right-hand side of the camera image. The system does not look for the hand anywhere else.
- Changing gestures takes about a second, so players need to show the next gesture a little before a junction. The direction buffer helps, but only partly.
- The live evaluation was small: one webcam in one room. It shows that the system works, but it does not measure accuracy across many people; the offline evaluation on unseen people is the better guide for that.
- Retraining the model or repeating the formal evaluation needs an NVIDIA CUDA GPU, as well as the dataset images, which have to be rebuilt from the source because they are not stored in the repository.

18. FUTURE ENHANCEMENTS

The following are ideas for future work. None of them is part of the current version.

- Add a "no gesture" class, trained on other hand shapes and empty scenes, so that the model itself can reject anything outside the four commands.
- Use a hand detector so that the player's hand can be anywhere in the frame instead of inside a fixed box.
- Train with simulated low-light noise and motion blur, or collect images in darker rooms, to make the model more reliable in poor conditions.
- Run a proper live study with more people, webcams and rooms to measure real-world accuracy and response time.
- Make inference cheaper, for example by quantising the model, so that it runs better on computers without a GPU.

- Try a model that looks at several frames together, which could recognise gesture changes faster and more reliably than fixed smoothing.

19. CONCLUSION

We set out to build a game that can be played with hand gestures alone, and the finished application does exactly that: a CNN watches the player's hand through a webcam and steers a Pac-Man-style maze game. The dataset of 2,000 hand images was built from HaGRID with a split that can be reproduced exactly, and a MobileNetV2 network pretrained on ImageNet was adapted to the four gestures through two-stage transfer learning. The model reached 99.5% validation accuracy, 99.0% test accuracy with a macro F1-score of 0.990, and 99.6% accuracy on people it had never seen.

Running recognition on its own thread, behind a confidence threshold and temporal smoothing, let the game hold 60 frames per second while the camera was processed at about 30. Live trials and automated tests confirmed that the gestures control the game reliably and that uncertain input leads to no command rather than a wrong one. The project shows that a compact CNN, trained with transfer learning on a modest dataset, is enough for practical real-time game control. Its weak points are clear as well: the lack of a reject class for unsupported gestures and the sensitivity to low light and motion blur are where further work would pay off most.

20. REFERENCES

[1] A. Kapitanov, K. Kvanchiani, A. Nagaev, R. Kraynov and A. Makhliarchuk, "HaGRID: HAnd Gesture Recognition Image Dataset," in *Proc. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024. arXiv:2206.08219.

[2] C. Mills, "hagrid-sample-500k-384p," Hugging Face Datasets. [Online]. Available: https://huggingface.co/datasets/cj-mills/hagrid-sample-500k-384p

[3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510-4520.

[4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248-255.

[5] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8024-8035.

[6] TorchVision maintainers and contributors, "TorchVision: PyTorch's Computer Vision Library," 2016. [Online]. Available: https://github.com/pytorch/vision

[7] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[8] Pygame Community, "Pygame 2.6.1." [Online]. Available: https://www.pygame.org

[9] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. International Conference on Learning Representations (ICLR)*, 2019.

[10] P. Micikevicius et al., "Mixed Precision Training," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[11] C. R. Harris et al., "Array Programming with NumPy," *Nature*, vol. 585, pp. 357-362, 2020.

[12] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.

[13] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[14] Creative Commons, "Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)." [Online]. Available: https://creativecommons.org/licenses/by-sa/4.0/

[15] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. International Conference on Machine Learning (ICML)*, 2015, pp. 448-456.

[16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929-1958, 2014.

[17] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209-212, 1927.